РОЗРОБКА ЗАСТОСУНКІВ ЗІ ШТУЧНИМ ІНТЕЛЕКТОМ

Практична робота №3. Інтеграція великої мовної моделі через API

Мета: Підключити готову мовну модель через API й побудувати на ній текстовий сценарій: передати системну інструкцію, дані користувача та додатковий контекст, винести роботу з моделлю в окремий модуль, опрацювати збої, таймаути й ліміти API, винести конфігурацію моделі назовні та порівняти результати за різних параметрів генерації.

Зміст

1. Теоретичні відомості
2. Завдання до виконання
3. Критерії приймання
4. Контрольні запитання
5. Рекомендована література

1. Теоретичні відомості

У ПР1 «двигуном» був звичайний REST API, у ПР2 — модель, що працювала локально. Тут поєднуються обидві лінії: модель знову стає **зовнішнім ресурсом**, до якого застосунок звертається через мережу, але це вже мовна модель, і поводиться вона інакше за звичайний сервіс.

Опора, щоб виконувати роботу свідомо.

Чим виклик LLM відрізняється від звичайного API. Звичайний ендпоінт на однаковий запит повертає однакову відповідь, відповідає за десятки мілісекунд і коштує стільки ж, скільки будь-який інший. Мовна модель порушує всі три припущення: відповідь **щоразу може бути іншою**, чекати доводиться секунди, а платня нараховується за обсяг тексту. Тому код, написаний за звичкою «викликав і отримав», на LLM ламається.

Ролі повідомлень. Запит до моделі — це не один рядок, а послідовність повідомлень із ролями. **System** задає, ким модель має бути і як поводитися; **user** містить те, що надійшло від користувача; **assistant** — попередні відповіді моделі, якщо ви ведете діалог. Розділення ролей має практичний сенс: інструкція розробника й дані користувача не змішуються в один текст, і користувач не може випадково (чи навмисно) переписати правила.

Токени. Модель працює не з символами й не зі словами, а з **токенами** — шматками тексту. За них рахують і вартість, і обмеження: у моделі є **контекстне вікно** — максимальна кількість токенів, яка вміщується в запит разом із відповіддю. Груба орієнтовна оцінка для української мови — приблизно один токен на два-три символи. Звідси два наслідки: довгий контекст коштує грошей на кожному виклику, а якщо запит із відповіддю не вміщується у вікно, відповідь обірветься на півслові.

Параметри генерації. Найважливіші три. **Temperature** керує тим, наскільки модель схильна обирати менш очевидні продовження: нижча — відповіді передбачуваніші й одноманітніші, вища — різноманітніші й менш надійні. **Top-p** обмежує вибір найімовірнішими варіантами.

Max tokens обмежує довжину відповіді — і саме він найчастіше винен, коли відповідь обривається. Для задач, де потрібна стабільність, температуру знижують.

Що ламається на практиці. Мережа й провайдер дають власний набір збоїв: таймаут, коли модель думає надто довго; **429** — перевищено ліміт запитів; **5xx** — тимчасова недоступність сервісу; помилка автентифікації через невірний або протермінований ключ. Окремий випадок — відповідь прийшла, але обірвана лімітом токенів. Кожен із них треба відрізняти й обробляти по-різному: на 429 має сенс зачекати й повторити, на невірний ключ — ні.

Ключ доступу — це секрет. Він не потрапляє в код, у репозиторій і в журнали. Стандартний спосіб — змінні середовища й файл, який не комітять. Ключ, випадково опублікований у репозиторії, вважається скомпрометованим.

Навіщо порівнювати. Той самий сценарій із різними параметрами або на різній моделі дає помітно різний результат — за якістю, стабільністю, швидкістю й вартістю. Поки порівняння не зроблено на власних прикладах, вибір параметрів лишається здогадкою. Це продовження думки з ЛК2: рішення ухвалюють за замірами, а не за загальними міркуваннями.

2. Завдання до виконання

Зробіть веб-застосунок **помічника служби підтримки інтернет-магазину**, який відповідає на звернення клієнтів **на підставі правил магазину**. Користувач вводить звернення довільним текстом і отримує відповідь, а разом із нею — час виконання й назву моделі, що відповідала.

Правила лежать у файлі `context.md` з матеріалів роботи: доставка, оплата, повернення, гарантія, графік підтримки. Помічник має відповідати **тільки за ними**:

- якщо відповідь у правилах є — дати її своїми словами, зрозуміло для клієнта;
- якщо відповіді немає — прямо сказати про це й **не вигадувати**;
- якщо звернення можна зрозуміти по-різному — не гадати, а уточнити.

Правила є даними, а не частиною коду: заміна `context.md` на інший файл має змінювати поведінку помічника **без зміни програми**.

Модель викликається **через API**; роботу з нею винесіть в окремий модуль, як це було зроблено в ПР2. Бекенд — на FastAPI.

Спосіб реалізації — на ваш розсуд (самостійно або скеровуючи AI-асистента), але ви маєте **розуміти й уміти пояснити** кожне рішення.

Матеріали. В архіві до цієї роботи, папка `pr3`, вас чекає підготовлене середовище: `requirements.txt`, скрипт перевірки середовища, каркас застосунку з заготовками та файл із прикладом конфігурації. Каркас дає структуру проєкту й веб-рівень; робота з моделлю, обробка збоїв і формування запиту — ваші.

Там само лежить `context.md` — короткий витяг із правил вигаданої організації. Це і є той додатковий контекст, який треба передавати моделі: підстава для відповіді, а не просто текст у запиті.

Де вести роботу. Розпакуйте архів у корінь власного репозиторію курсу — поруч із папками `pr1` і `pr2`. Свій код пишіть у папці роботи й комітьте частинами.

Доступ до моделі. У курсі використовується **Gemini** від Google. Ключ видається безкоштовно й одразу:

1. Відкрийте **Google AI Studio** — https://aistudio.google.com — і увійдіть зі своїм акаунтом Google.
2. Натисніть **Get API key**, далі **Create API key**.
3. Скопіюйте ключ у рядок `LLM_API_KEY` файлу `.env` (створюється копіюванням `.env.example`).

Ані банківська картка, ані обліковий запис Google Cloud не потрібні: безкоштовного рівня для цієї роботи достатньо. Адреса сервісу й назва моделі вже прописані в `.env.example`.

Безкоштовний рівень має **ліміти на кількість запитів** за хвилину й за добу. Це не завада, а частина завдання: саме на них ви побачите справжню помилку `429` і зможете перевірити, як її обробляє ваш застосунок.

Технічно Gemini доступний через **OpenAI-сумісний інтерфейс**, тож каркас користується звичайним клієнтом `openai` — змінюється лише адреса сервісу. Завдяки цьому перехід на іншого провайдера чи на локальний сервер моделей зводиться до зміни конфігурації, а не коду.

Орієнтири.

- Конфігурація моделі (адреса сервісу, назва моделі, ключ, параметри генерації) задається **ззовні** — через змінні середовища, а не константами в коді.
- Системна інструкція, звернення користувача й додатковий контекст передаються **окремими частинами** запиту, а не склеюються в один рядок.
- Час виконання вимірюється й показується — як і в ПР2.

Продумайте самостійно.

- Що має бути в системній інструкції, а що — у контексті? Де межа між «як поводитися» і «на підставі чого відповідати»?
- Як домогтися, щоб модель не вигадувала того, чого немає в правилах? Чим це перевірити?
- Які збої варто повторювати автоматично, а які — ні? Скільки разів і з якою паузою?
- Що показати користувачеві, коли сервіс недоступний? Чи має він побачити технічну помилку?
- Скільки чекати на відповідь, перш ніж перервати запит?
- Як не дати звертанню користувача переписати вашу системну інструкцію?

Порівняння. Підготуйте **5–7 звернень** різного характеру: типове, нетипове, з питанням поза межами правил, з некоректним або порожнім вводом. Прожене їх через застосунок **двічі** — за різних параметрів генерації (наприклад, низька й висока температура) або на двох різних моделях. Зафіксуйте, що змінилося: стабільність відповідей, їхня довжина, час виконання, схильність вигадувати. Короткі висновки запишіть у файл.

Цей набір звернень — продовження того, що ви почали збирати в ПР2, і основа для оцінювання якості в наступних роботах.

3. Критерії приймання

- Мовна модель викликається **через API**; робота з нею винесена в **окремий модуль**.
- Системна інструкція, дані користувача та додатковий контекст передаються **окремими частинами** запиту.

- Помічник відповідає **на підставі правил** із `context.md`, а на питання поза їхніми межами **не вигадує** відповідь.
- Заміна `context.md` змінює поведінку помічника **без зміни коду**.
- Конфігурація моделі задається **ззовні**; ключ доступу **не потрапив** ані в код, ані в репозиторій.
- Реалізовано обробку збоїв: **таймаут**, **перевищення лімітів**, **недоступність сервісу**, **помилка автентифікації** — з відповідними HTTP-статусами.
- **Час виконання** вимірюється й показується; використана модель відображається у відповіді.
- Проведено **порівняння** щонайменше двох конфігурацій на 5–7 підготовлених зверненнях; висновки зафіксовано.
- Ви можете **пояснити** кожне ухвалене рішення.

4. Контрольні запитання

1. Чим виклик мовної моделі відрізняється від виклику звичайного REST API?
2. Навіщо розділяти системну інструкцію й повідомлення користувача, якщо модель усе одно бачить обидва?
3. Що таке токен і що таке контекстне вікно? Чому через них обривається відповідь?
4. На що впливає температура і в яких задачах її знижують?
5. Які збої може дати виклик LLM і які з них має сенс повторювати?
6. Чому ключ доступу не можна тримати в коді і що робити, якщо він потрапив у репозиторій?
7. Що змінилося у відповідях при зміні параметрів генерації і як ви це встановили?

5. Рекомендована література

1. Huyen C. AI Engineering: Building Applications with Foundation Models. Sebastopol: O'Reilly Media, 2025. 496 p. ISBN 978-1-0981-6630-4.
2. OpenAI. API Reference — Chat Completions. URL: https://platform.openai.com/docs/api-reference/chat (дата звернення: 01.09.2026).
3. FastAPI Documentation. URL: https://fastapi.tiangolo.com/ (дата звернення: 01.09.2026).
4. OWASP. Top 10 for Large Language Model Applications. URL: https://owasp.org/www-project-top-10-for-large-language-model-applications/ (дата звернення: 01.09.2026).